

Informe Técnico

Proyecto Google Ads Analytics

Clasificación de Rentabilidad de Campañas: Preprocesamiento, Modelado y Evaluación

SCY1101 — Programación para la Ciencia de Datos

Proyecto	Google Ads Analytics — Clasificación Binaria de Rentabilidad
Dataset	Google Ads Sales Dataset (Kaggle)
Repositorio	github.com/carlos31255/google_ads_analytics

1. Resumen Ejecutivo

El presente informe documenta el proceso completo del proyecto: análisis exploratorio (EDA), limpieza y transformación de datos, optimización de hiperparámetros con Optuna, entrenamiento del modelo final y evaluación de rendimiento. El objetivo es predecir si una campaña publicitaria de Google Ads será rentable (`Is_Profitable`), problema tratado como clasificación binaria supervisada.

Variable Objetivo

$Profit_Margin = (Sale_Amount - Cost) / Cost$

$Is_Profitable = 1$ si $Profit_Margin \geq \tau$, 0 en caso contrario.

Se aplica un τ dinámico (cuantil 70 del margen) para mantener una distribución útil de clases (70% / 30%).

Resultados Clave del Proyecto Completo

Métrica	Valor
Filas originales	2,600
Filas supervisadas (train + test)	2,366
Features finales	22
Modelo ganador	LightGBM
F1-Macro en CV (Optuna)	0.9694
Accuracy en test	97.47%
F1-Macro en test	0.9700

Métrica	Valor
ROC-AUC	Implícito en curva ROC
Trials de Optuna ejecutados	30

2. Análisis Exploratorio Inicial

2.1 Descripción General del Dataset

El dataset `GoogleAds_DataAnalytics_Sales_Uncleaned.csv` contiene registros de anuncios individuales de una campaña de Marketing Digital para un curso de Data Analytics. Cada fila representa un anuncio con sus métricas de rendimiento.

Dimensiones originales: 2,600 filas × 13 columnas

Columna	Tipo original	Descripción
Ad_ID	object	Identificador único del anuncio
Campaign_Name	object	Nombre de la campaña (con typos)
Clicks	float64	Número de clics recibidos
Impressions	float64	Número de impresiones del anuncio
Cost	object	Costo del anuncio (formato \$XXX.XX)
Leads	float64	Número de leads generados
Conversions	float64	Número de conversiones
Conversion Rate	float64	Tasa de conversión (Conversions / Clicks)
Sale_Amount	object	Monto de venta (formato \$X,XXX)
Ad_Date	object	Fecha del anuncio (formatos múltiples)
Location	object	Ciudad objetivo (con variantes)
Device	object	Dispositivo (con variantes de capitalización)
Keyword	object	Palabra clave del anuncio

2.2 Problemas de Calidad Detectados

A. Valores Faltantes (NaN)

Columna	Nulos	Porcentaje
Conversion Rate	626	24.08%
Sale_Amount	139	5.35%
Clicks	112	4.31%
Cost	97	3.73%
Conversions	74	2.85%

Columna	Nulos	Porcentaje
Impressions	54	2.08%
Leads	48	1.85%

B. Columnas Monetarias Codificadas como Texto

`Cost` y `Sale_Amount` almacenadas como strings tipo '\$231.88' y '\$1,892'

C. Inconsistencias de Texto (Typos y Capitalización)

Columna	Variantes encontradas	Valor correcto
Campaign_Name	4 variantes	1 campaña única
Location	'hyderabad', 'HYDERABAD', 'Hyderbad', 'hydrebad'	hyderabad
Device	9 variantes de 3 dispositivos	3 dispositivos únicos

D. Fechas con Formatos Heterogéneos

La columna `Ad_Date` presentó tres formatos: `YYYY-MM-DD`, `DD-MM-YYYY`, `YYYY/MM/DD`.

2.3 Distribución de la Variable Objetivo

Is_Profitable	Frecuencia	Porcentaje
1 (Rentable)	710	30.0%
0 (No rentable)	1,656	70.0%

Los 234 registros con `Cost` o `Sale_Amount` nulos fueron excluidos del set supervisado para evitar etiquetas artificiales.

3. Metodología de Transformación

3.1 Arquitectura del Pipeline de Preprocesamiento

```
Datos Crudos (2,600 × 13)
|
|-- [Target] Is_Profitable por margen en crudo + selección de tau dinámico
|-- [Target] Exclusión de registros con target desconocido (234 filas)
|
|-- [A] DateStandardizerTransformer → estandariza fechas + extrae features temporales
|-- [B] TextNormalizerTransformer → normaliza texto con fuzzy matching
|-- [C] DropColumnsTransformer → elimina columnas de leakage
|-- [D] MonetaryCleanerTransformer → convierte '$X,XXX' → float
```

```

|-- [E] DropHighMissingTransformer      → elimina columnas > 80% nulos
|-- [F] SmartImputerTransformer         → imputa según porcentaje de nulos
|-- [G] ColumnTransformer
    |-- num_pipe: [OutlierCapper → DropZeroVariance → StandardScaler]
    |-- cat_pipe: [OneHotEncoder]
|
Datos Procesados Supervisados (2,366 × 22 features)

```

3.2 Transformers Implementados (8 clases sklearn)

Transformer	Problema que resuelve	Técnica
DateStandardizerTransformer	Fechas en múltiples formatos	Parseo secuencial, extrae month, day_of_week, is_weekend
TextNormalizerTransformer	Typos y capitalización en texto	difflib.get_close_matches() con umbral 0.6
DropColumnsTransformer	Data Leakage directo	Eliminación de Ad_ID, Cost, Sale_Amount, Ad_Date
MonetaryCleanerTransformer	Monetarios como strings	Regex + conversión a float
DropHighMissingTransformer	Columnas con >80% nulos	Umbral configurable (salvaguarda)
SmartImputerTransformer	1.85% – 24.08% de nulos	Mediana (<10%), fallback mediana (>10%)
OutlierCapper	Valores atípicos extremos	Recorte IQR [Q1-1.5·IQR, Q3+1.5·IQR]
OneHotEncoder	Variables categóricas	Dummies con handle_unknown='ignore'

3.3 Prevención de Data Leakage

- **Punto 1:** La creación de Is_Profitable ocurre antes del pipeline, sobre datos monetarios crudos sin imputar.
- `Cost` y `Sale_Amount` se eliminan antes de que el ColumnTransformer las procese.
- El escalado y la imputación aprenden exclusivamente de `X_train`, nunca de `X_test`.
- La columna objetivo se separa como y antes de pasar X al pipeline.

3.4 Módulos de Soporte

- **audit.py:** Verificación SHA-256 del dataset crudo contra metadata.json. Detecta modificaciones accidentales. Se normalizan saltos de línea (CRLF→LF) antes del hash para compatibilidad cross-platform.
- **optimization.py:** Downcasting de tipos (int64→int8/16/32, float64→float32). Reducción de memoria: 1.43 MB → 1.38 MB (3.5%).

4. Optimización de Hiperparámetros con Optuna

4.1 Estrategia de Búsqueda

Se utilizó **Optuna** con el sampler `TPESampler` (semilla 42) para explorar simultáneamente el espacio de algoritmos y sus hiperparámetros en 30 trials. Cada trial construye un pipeline [`PCA` opcional → `Clasificador`] y lo evalúa con **Cross-Validation estratificada de 3 folds** usando **F1-Macro** como métrica objetivo.

Espacio de búsqueda:

Dimensión	Opciones
Reducción dimensional	PCA (varianza 0.85/0.90/0.95/0.99) o sin PCA
Algoritmos	SVM, RandomForest, XGBoost, LightGBM, LogisticRegression
Hiperparámetros propios	C, n_estimators, max_depth, learning_rate (según algoritmo)

4.2 Resultado Ganador

Parámetro	Valor
Algoritmo	LightGBM
use_pca	False
lgb_n_estimators	200
lgb_learning_rate	0.2840
F1-Macro CV	0.9694

El estudio completo se persiste en `models/trained_models/optuna_study.db` (SQLite) para permitir visualizaciones interactivas en el notebook `04_hyperparameter_optimization.ipynb` (historial de optimización, importancia de parámetros, coordenadas paralelas).

4.3 Justificación de LightGBM como Ganador

LightGBM es particularmente efectivo en datasets tabulares con desbalance de clases (70/30) gracias al parámetro `class_weight='balanced'` y a su algoritmo de boosting basado en histogramas que evita el sobreajuste con muchos estimadores. La ausencia de PCA en la configuración ganadora indica que las 22 features originales aportan suficiente señal sin reducción dimensional.

5. Entrenamiento del Modelo Final

Con los hiperparámetros ganadores de Optuna, se entrenó el pipeline final sobre el conjunto completo de entrenamiento (`X_train`, `y_train`):

```
Pipeline:
└─ classifier: LGBMClassifier(
    n_estimators=200,
    learning_rate=0.2840,
    class_weight='balanced',
    random_state=42
)
```

El modelo entrenado se serializa en `models/trained_models/final_classifier.joblib` para reutilización sin re-entrenamiento.

6. Evaluación del Modelo

6.1 Reporte de Clasificación (Test Set — 474 muestras)

Clase	Precision	Recall	F1-Score	Support
No Rentable (0)	0.9848	0.9789	0.9819	332
Rentable (1)	0.9514	0.9648	0.9580	142
Accuracy			0.9747	474
Macro avg	0.9681	0.9719	0.9700	474
Weighted avg	0.9748	0.9747	0.9747	474

6.2 Interpretación de Métricas

- **Accuracy (97.47%)**: El modelo clasifica correctamente 462 de 474 anuncios del test.
- **F1-Macro (0.9700)**: Métrica principal del proyecto. Al promediar las clases sin ponderar, penaliza el rendimiento en la clase minoritaria (Rentable). Valor muy alto indica que el modelo no colapsa hacia la clase mayoritaria.
- **Precision en clase Rentable (0.9514)**: De cada 100 anuncios predichos como rentables, ~95 lo son realmente. Baja tasa de falsos positivos.
- **Recall en clase Rentable (0.9648)**: El modelo detecta el 96.5% de los anuncios verdaderamente rentables. Bajo número de anuncios rentables perdidos.
- **Manejo del desbalance (70/30)**: El uso de `class_weight='balanced'` fue determinante. Sin él, un clasificador trivial podría obtener 70% de accuracy simplemente prediciendo siempre clase 0.

6.3 Visualizaciones Generadas

- `results/plots/confusion_matrix.png` — Matriz de confusión: distribución visual de aciertos y errores por clase.
- `results/plots/roc_curve.png` — Curva ROC: capacidad de separación de clases del modelo LightGBM.

- `results/plots/model_comparison_cv.png` — Comparación de F1-Macro entre los 5 modelos evaluados durante la búsqueda de Optuna.

7. Conclusiones y Recomendaciones

7.1 Conclusiones

- **El pipeline de preprocesamiento resultó robusto y reproducible.** Encapsular cada transformación en un transformer sklearn garantiza que los mismos pasos se apliquen de forma idéntica sobre datos nuevos, sin riesgo de leakage ni inconsistencias manuales.
- **Optuna demostró ser más eficiente que un GridSearch tradicional.** En 30 trials exploró simultáneamente 5 familias de algoritmos y sus hiperparámetros, convergiendo al mejor resultado (LightGBM) sin necesidad de búsqueda exhaustiva.
- **El modelo final alcanzó métricas de producción (F1-Macro 0.97).** Un rendimiento de esta magnitud en clasificación binaria con desbalance 70/30 indica que el pipeline de features y la elección del algoritmo fueron acertados.
- **La variable objetivo fue ingenierizada robustamente.** Al calcular `Is_Profitable` sobre valores monetarios crudos con un tau dinámico, se obtuvo una distribución operativa (70/30) evitando el colapso de clase observado con umbrales fijos.

7.2 Dificultades Encontradas

- **Formatos de fecha múltiples:** Requirieron parseo secuencial con doble pasada.
- **Columnas monetarias como strings:** Necesitaron un transformer dedicado antes del pipeline numérico.
- **Colapso de clase con umbrales fijos:** Resuelto con fallback dinámico (cuantil 70 del margen).
- **Verificación de integridad cross-platform:** CRLF/LF en Windows alteraba el hash SHA-256; resuelto normalizando bytes antes del cálculo.
- **Duplicación del archivo `optuna_study.db`:** Se generó un segundo archivo en `models/` al ejecutar Optuna fuera del script principal. El archivo canónico es `models/trained_models/optuna_study.db`.

7.3 Recomendaciones

- **Mejorar la imputación de Conversion Rate:** Reemplazar el fallback por KNNImputer o IterativeImputer (24% de nulos).
- **Ampliar el espacio de búsqueda de Optuna:** Aumentar a 50-100 trials e incluir hiperparámetros adicionales de LightGBM (`num_leaves`, `min_child_samples`).
- **Añadir tests unitarios:** Implementar pruebas con pytest para cada transformer.
- **Eliminar el `optuna_study.db` duplicado** de `models/` y añadirlo al `.gitignore`.
- **Mantener trazabilidad del target:** Versionar el tau utilizado en cada corrida para garantizar reproducibilidad del conjunto supervisado.

Anexo A — Estructura del Proyecto

```
google_ads_analytics/
├── data/
│   ├── raw/
│   │   ├── GoogleAds_DataAnalytics_Sales_Uncleaned.csv
│   │   └── metadata.json          # Hash SHA-256 para integridad
│   └── processed/
│       ├── GoogleAds_Processed.csv      # Dataset limpio y transformado
│       ├── X_train.csv / X_test.csv     # Sets de entrenamiento y prueba
│       └── preprocessing_pipeline.joblib # Pipeline serializado
├── models/
│   └── trained_models/
│       ├── best_params.pkl            # Hiperparámetros ganadores de Optuna
│       ├── final_classifier.joblib     # Modelo LightGBM entrenado
│       └── optuna_study.db             # Historial completo de trials
├── notebooks/
│   ├── 01_exploratory_analysis.ipynb
│   ├── 02_Pipelines.ipynb
│   ├── 03_unsupervised_modeling.ipynb
│   ├── 04_hyperparameter_optimization.ipynb
│   ├── 05_model_evaluation.ipynb
│   └── 06_final_analysis.ipynb
├── src/
│   ├── __init__.py
│   ├── audit.py
│   ├── transformers.py
│   ├── pipeline.py
│   ├── optimization.py
│   ├── hyperparameter_tuning.py
│   ├── model_training.py
│   └── model_evaluation.py
├── results/
│   ├── metrics/classification_report.csv
│   └── plots/
│       ├── confusion_matrix.png
│       ├── roc_curve.png
│       └── model_comparison_cv.png
├── main.py
└── requirements.txt
```


Anexo B — Dependencias

Librería	Versión	Uso
pandas	3.0.3	Manipulación de datos
numpy	2.4.6	Operaciones numéricas
scikit-learn	1.8.0	Pipeline, transformers, métricas
matplotlib	3.10.9	Visualizaciones
seaborn	0.13.2	Visualizaciones estadísticas
optuna	4.8.0	Optimización bayesiana de hiperparámetros
xgboost	3.2.0	Algoritmo XGBoost
lightgbm	4.6.0	Algoritmo LightGBM (modelo ganador)
jobjlib	1.5.3	Serialización de modelos
plotly	6.7.0	Visualizaciones interactivas de Optuna
diffliB	stdlib	Fuzzy matching de texto
hashlib	stdlib	SHA-256 para auditoría